



Music Playlist Manager

Presentazione Terza Sprint



Gruppo 01

0622702428 – Domenico Nappo

IE22700035 – Arianna Paletta

IE22700189 – Vittorio Postiglione

IE22700199 – Mattia Sanzari

IE22700027 – Debora Villano

DIAGRAMMA DELLE CLASSI

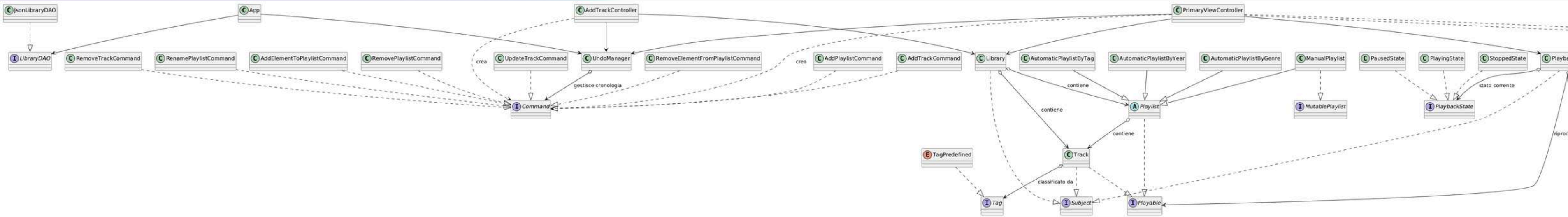


DIAGRAMMA DELLE CLASSI

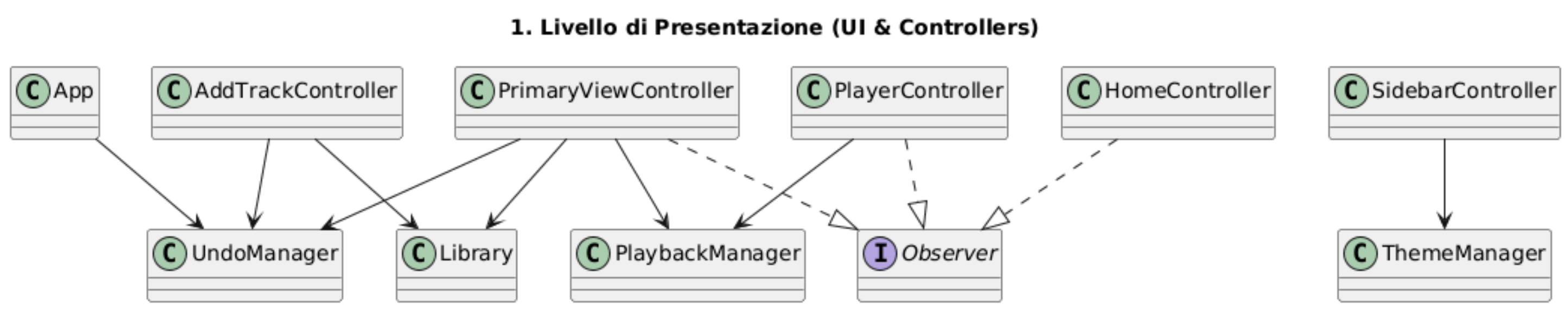
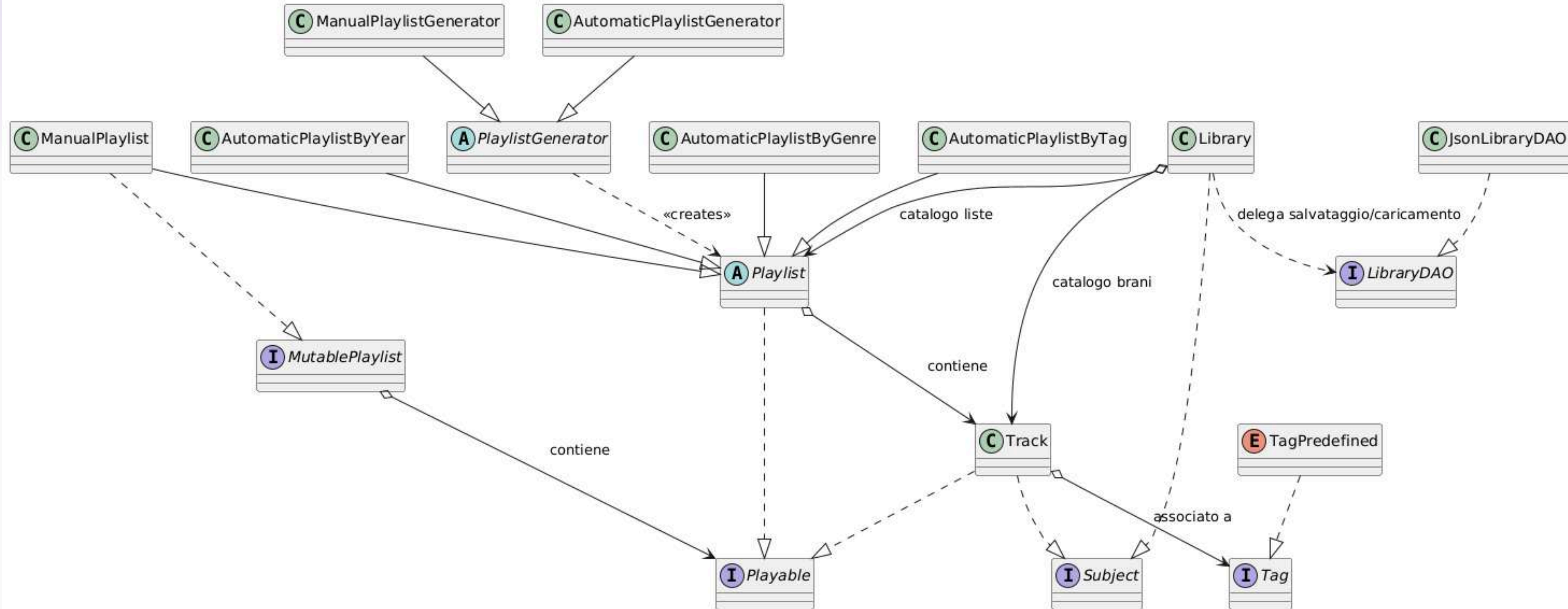


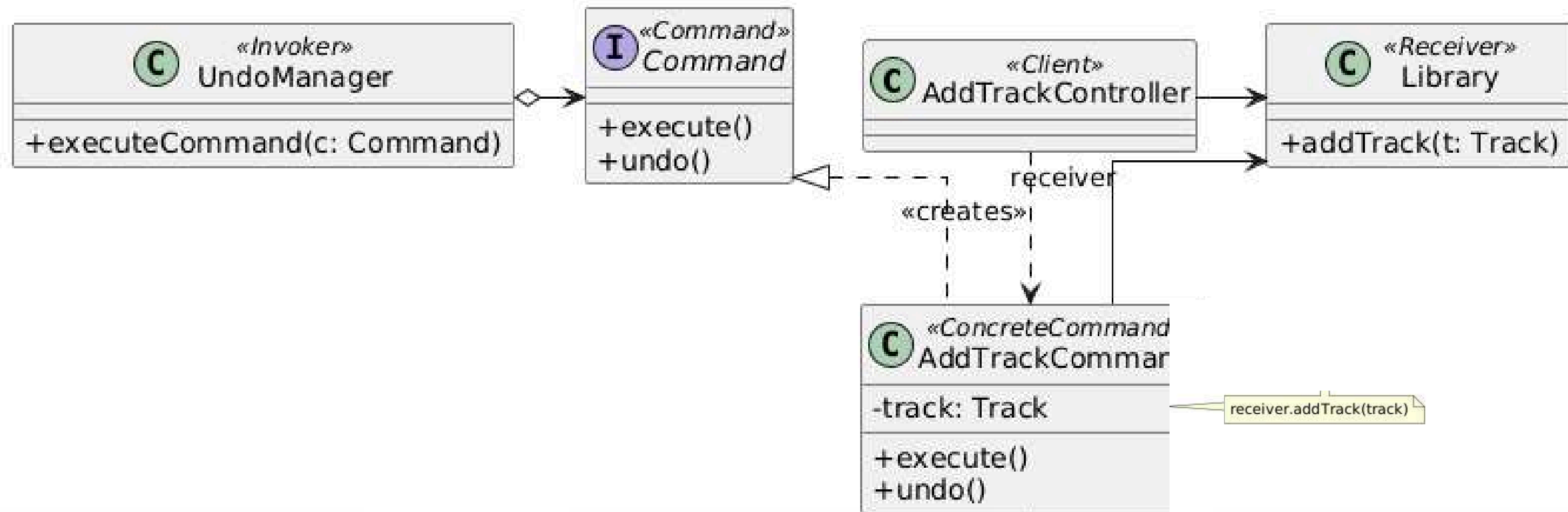
DIAGRAMMA DELLE CLASSI

3. Livello del Dominio (Strutture Dati, Playlist e Persistenza)



Pattern Command - Gestione dell'Undo

Il Command trasforma una richiesta o un'azione in un oggetto indipendente.



SUPPORTO NATIVO PER UNDO E REDO

Ogni **modifica** (aggiungere un brano, rinominare una playlist, ecc.) è **racchiusa** in un **oggetto** specifico dotato di un metodo **undo()** → **L'UndoManager** può impilare questi oggetti e "tornare indietro nel tempo" annullandoli uno a uno.

FORTE DISACCOPIAMENTO

Chi **scatena l'azione** (es. il bottone premuto nel **AddTrackController**) non deve conoscere le **complesse dinamiche** interne di chi la **esegue** materialmente (la **Library**). Il controller crea il comando, lo lancia e se ne dimentica.

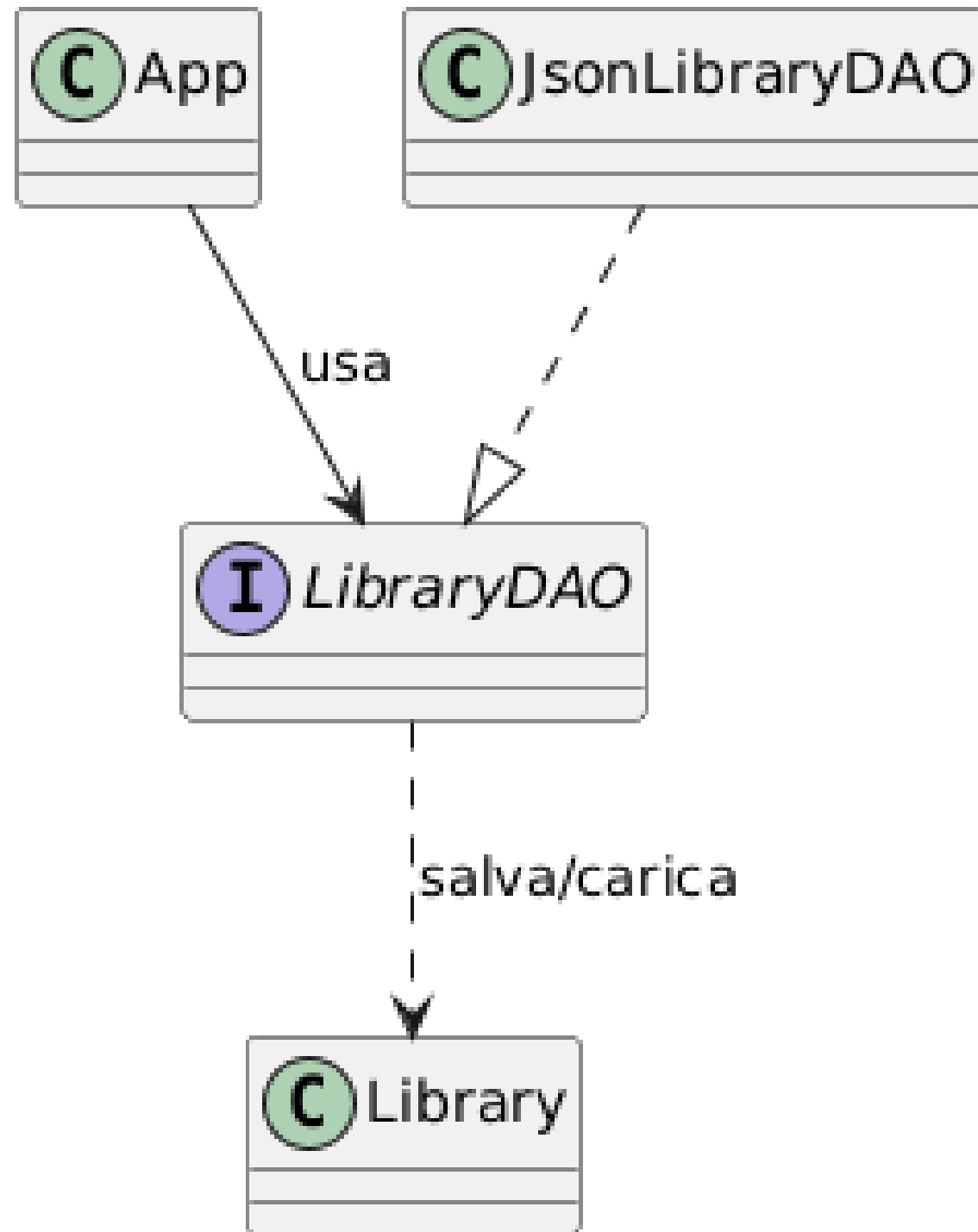
ESTENSIBILITÀ (PRINCIPIO OPEN/CLOSED):

L'aggiunta di nuove funzionalità, come un **comando** di **pulizia** libreria, non richiede **modifiche** ai **controller** esistenti: implementando una classe **dedicata**, ad esempio **ClearLibraryCommand**, si preserva la **pulizia dell'architettura**.

Pattern DAO - Persistenza dei dati

Il DAO serve a gestire l'accesso ai dati (salvataggio e caricamento) in modo pulito e indipendente.

Pattern Architetturale DAO (Data Access Object)



SEPARAZIONE DELLE RESPONSABILITÀ

La **logica** di **business** e quella di **persistenza** devono essere completamente **isolate**. Il **modello** (Library) gestisce esclusivamente la **gestione** della **musica**, rimanendo **indipendente** dalle modalità e dai luoghi di **archiviazione** dei dati.

ALTA FLESSIBILITÀ (INTERCAMBIABILITÀ)

L'utilizzo di un **JsonLibraryDAO** consente attualmente il **salvataggio** dei **dati** in formato **JSON**. In caso di migrazione verso un **database SQL** o un servizio di **archiviazione cloud**, è sufficiente implementare un nuovo **SqlLibraryDAO** per adattare il sistema **senza richiedere modifiche** al codice dell'applicazione esistente.

FACILITÀ DI TESTING

L'architettura adottata facilita notevolmente il **collaudo dell'applicazione**. Durante la fase di test, è possibile impiegare un **DAO** di tipo **"mock"** che archivia i dati nella memoria **RAM** anziché su **disco**, incrementando la **velocità** di **esecuzione** e garantendo l'isolamento dei processi di verifica.

Trello

The screenshot displays a Trello board with a Kanban workflow. The board is organized into seven columns, each representing a stage in the project lifecycle. The first five columns (Product Backlog, Sprint Backlog, Task To Do, Task In Progress, and To verify) are currently empty, each showing a count of 0 and a '+ Aggiungi una scheda' button. The 'Task complete' column contains six task cards, and the 'User Story Done' column contains five user story cards. Each card provides a detailed description of the task or user story, progress indicators (checkmarks and completion percentages), and status tags (e.g., 'High priority', 'DN', 'AP'). The bottom of the board features a navigation bar with tabs for 'Inbox', 'Planner', 'Bacheche' (selected), and 'Cambia bacheche'.

Product Backlog 0 ...
+ Aggiungi una scheda

Sprint Backlog 0 ...
+ Aggiungi una scheda

Task To Do 0 ...
+ Aggiungi una scheda

Task In Progress 0 ...
+ Aggiungi una scheda

To verify 0 ...
+ Aggiungi una scheda

Task complete 82 ...

- Task - Configurazione**
Task - Setup iniziale del progetto, build system e ambiente di test
3/3
- Task - Core**
Task 1.1.1 - Logica di business, modello Track e validazione
3/3 DN
- Task - GUI**
Task 1.1.2 - Form JavaFX di inserimento e gestione eventi
3/3 DV
- Task - Test**
Task 1.1.3 - Suite di Unit Test JUnit per inserimento brano
2/2 DN
- Task - Test**
Task 1.4.2 - test: JUnit e test manuali per visualizzazione dettagli
2/2 AP
- Task - Core**
Task 1.2.1 - Logica di aggiornamento brano in Library
4/4 DN
- + Aggiungi una scheda

User Story Done 25 ...

- High priority**
UE1 - Management and rappresent...
US 1.1 - Inserimento brano
8/8 1 1
- High priority**
UE1 - Management and rappresent...
US 1.2 - Modifica brano
7/7 1 1
- High priority**
UE1 - Management and rappresent...
US - 1.3 Eliminazione brano
7/7 3 3
- High priority**
UE1 - Management and rappresent...
US 1.4 - Visualizzazione dettagli brano
7/7 2 2
- High priority**
UE1 - Management and rappresent...
US 1.5 Creazione playlist
7/7 1 1
- High priority**
UE1 - Management and rappresent...
US 1.6 Modifica playlist
7/7 2 2
- + Aggiungi una scheda

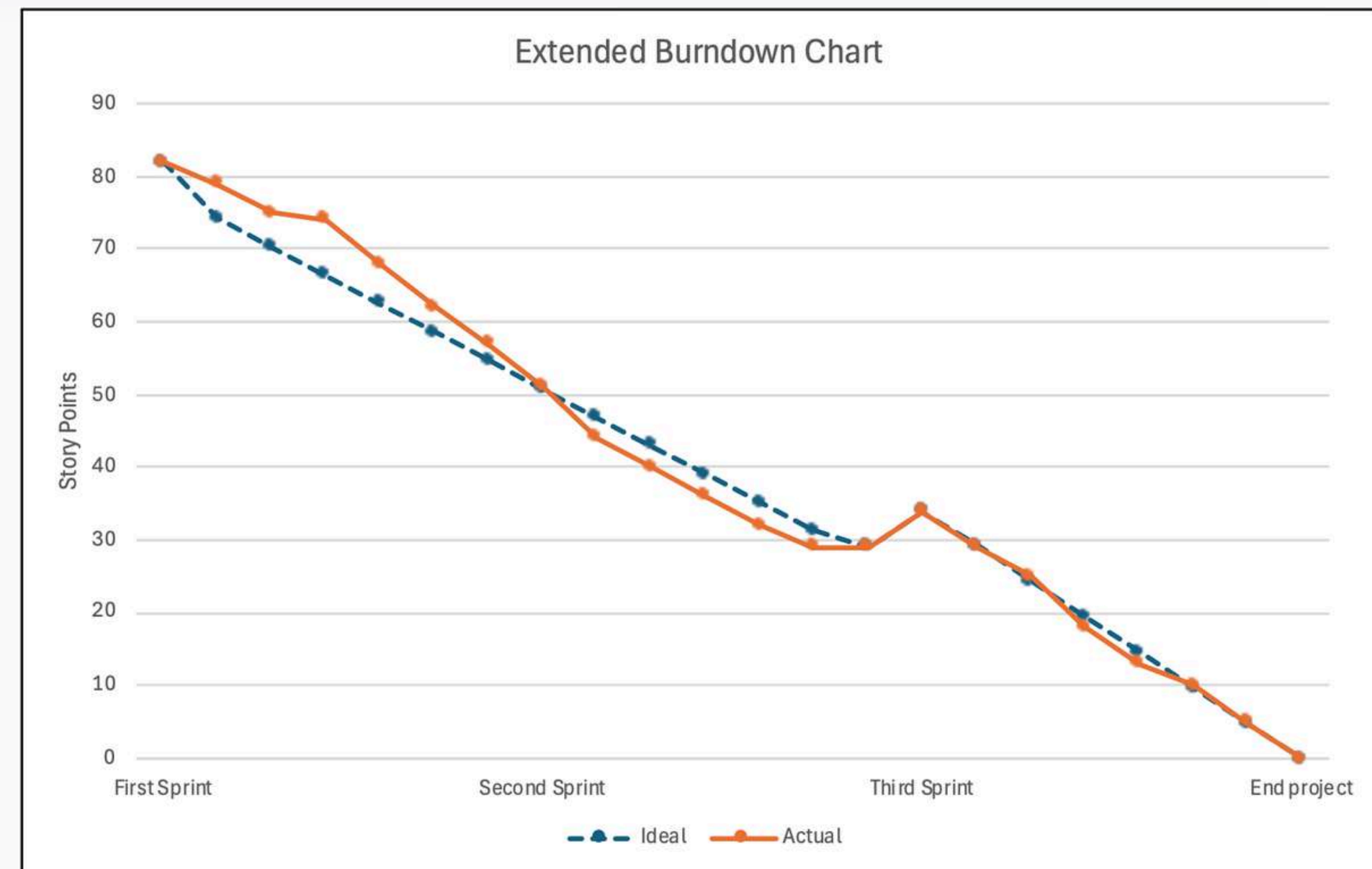
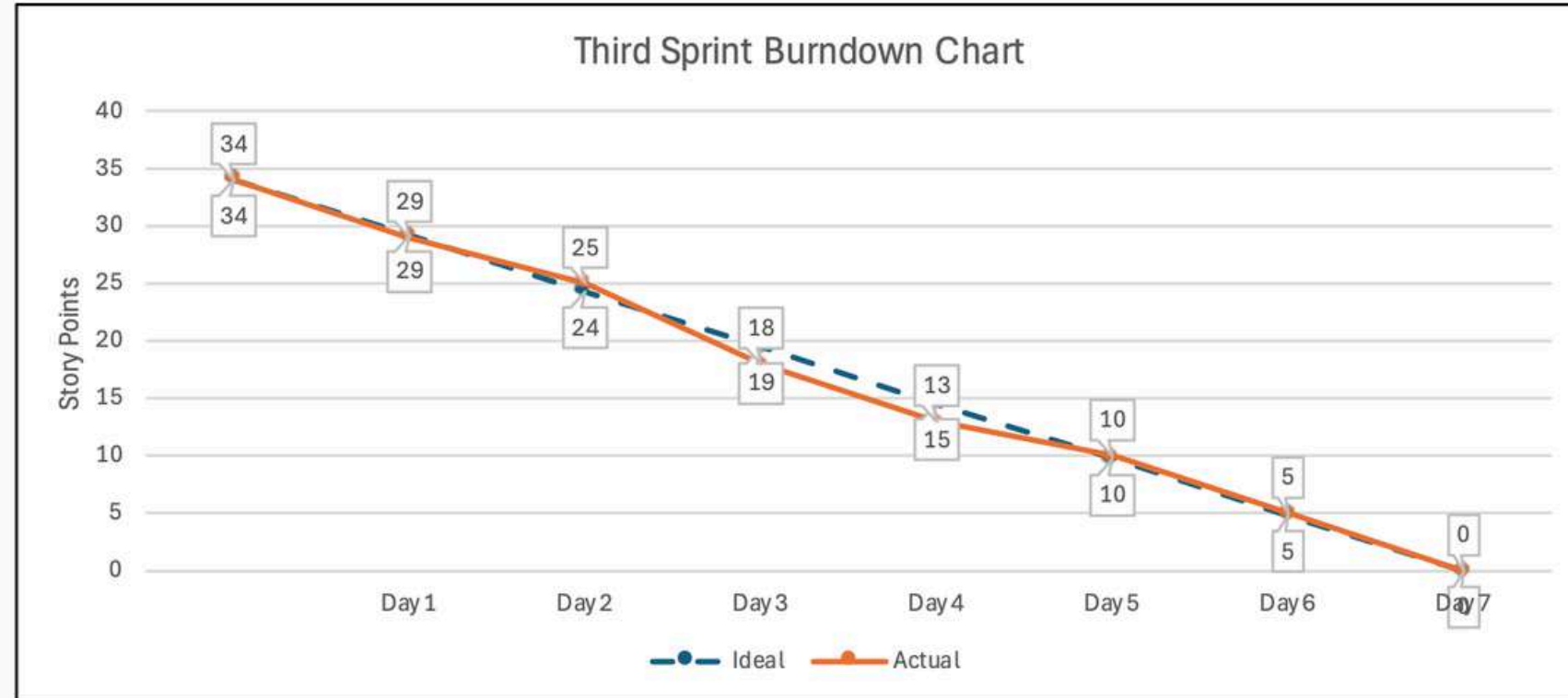
Navigation: Inbox Planner **Bacheche** Cambia bacheche

Sprint Backlog

ID User Story	Story Points	Nome Task	Area	Descrizione / Checklist	Stato	Assegnato A
US 4.1 - Undo operazione	8					
		Task 4.1.1 - Implementazione storico operazioni undo	CORE	Implementare la struttura necessaria per memorizzare le operazioni effettuate dall'utente, come aggiunta e rimozione di brani o playlist. Il sistema deve conservare l'ordine cronologico delle operazioni per permettere l'annullamento a ritroso.	Fatto	ARIANNA PALETTA
		Task 4.1.2 - Implementazione logica di annullamento	CORE	Implementare la logica che permette di annullare l'ultima operazione effettuata, ripristinando correttamente lo stato precedente della libreria o della playlist interessata.	Fatto	ARIANNA PALETTA
		Task 4.1.3 - Integrazione pulsante Undo nell'interfaccia	GUI	Aggiungere il comando Undo nell'interfaccia grafica e collegarlo alla logica del modello, assicurando che l'utente possa annullare le operazioni in modo semplice e immediato.	Fatto	VITTORIO POSTIGLIONE
		Task 4.1.4 - Test e validazione Undo	TEST	Verificare che il sistema annulli correttamente una o più operazioni in ordine inverso, gestendo anche il caso in cui non siano presenti operazioni da annullare.	Fatto	DEBORA VILLANO
US 4.2 - Tracce più riprodotte	5					
		Task 4.2.1 - Implementazione conteggio riproduzioni tracce	CORE	Implementare un meccanismo per registrare il numero di riproduzioni associate a ciascun brano ogni volta che viene avviata la riproduzione.	Fatto	DEBORA VILLANO
		Task 4.2.2 - Visualizzazione tracce più riprodotte	GUI	Aggiornare la home page o la sezione dedicata per mostrare l'elenco delle tracce più riprodotte, ordinate in base al numero di ascolti.	Fatto	VITTORIO POSTIGLIONE
		Task 4.2.3 - Test e validazione analytics tracce	TEST	Verificare che il conteggio delle riproduzioni venga aggiornato correttamente e che l'interfaccia mostri solo le tracce effettivamente riprodotte.	Fatto	DEBORA VILLANO
US 4.3 - Playlist più riprodotte	5					
		Task 4.3.1 - Implementazione conteggio riproduzioni playlist	CORE	Implementare un meccanismo per registrare il numero di riproduzioni associate alle playlist quando l'utente avvia il playback di una playlist.	Fatto	ARIANNA PALETTA
		Task 4.3.2 - Visualizzazione playlist più riprodotte	GUI	Aggiornare la home page o la sezione dedicata per mostrare le playlist più riprodotte, ordinate in base al numero di avvii della riproduzione.	Fatto	VITTORIO POSTIGLIONE
		Task 4.3.3 - Test e validazione analytics playlist	TEST	Verificare che il conteggio delle playlist più riprodotte sia corretto e che il sistema non mostri risultati quando nessuna playlist è stata ancora riprodotta.	Fatto	DEBORA VILLANO
US 5.2 - Generazione playlist per etichette	3					
		Task 5.2.1 - Implementazione generazione playlist per tag	CORE	Implementare la funzionalità che permette di generare automaticamente una playlist contenente tutte le tracce associate a uno specifico tag selezionato dall'utente.	Fatto	MATTIA SANZARI
		Task 5.2.2 - Interfaccia generazione playlist per etichette	GUI	Aggiungere nell'interfaccia la possibilità di selezionare un tag e avviare la generazione automatica della playlist corrispondente.	Fatto	DEBORA VILLANO
		Task 5.2.3 - Test e validazione playlist per tag	TEST	Verificare che la playlist generata contenga esclusivamente i brani associati al tag scelto e che il sistema gestisca correttamente il caso in cui non siano presenti brani corrispondenti.	Fatto	DEBORA VILLANO
US 6.1 - Riordinamento playlist durante la riproduzione	8					
		Task 6.1.1 - Implementazione riordinamento brani playlist	CORE	Implementare la logica necessaria per modificare l'ordine dei brani all'interno di una playlist, mantenendo la coerenza della struttura dati.	Fatto	DOMENICO NAPPO
		Task 6.1.2 - Gestione riordinamento durante il playback	CORE	Assicurare che il riordinamento dei brani futuri non interrompa la traccia attualmente in riproduzione e che il player prosegua seguendo il nuovo ordine aggiornato.	Fatto	DOMENICO NAPPO
		Task 6.1.3 - Interfaccia per spostamento brani	GUI	Implementare i controlli grafici necessari per permettere all'utente di cambiare la posizione dei brani nella playlist, ad esempio tramite pulsanti o azioni di spostamento.	Fatto	DOMENICO NAPPO
		Task 6.1.4 - Test e validazione riordinamento dinamico	TEST	Verificare che il riordinamento funzioni sia a player fermo sia durante la riproduzione, senza generare interruzioni o incoerenze nella coda.	Fatto	DOMENICO NAPPO
US 3.3 - Ricerca dei brani nella libreria musicale	1					
		Task 3.3.1 - Implementazione ricerca brani	GUI/CORE	Implementare un campo di ricerca che permetta di filtrare rapidamente i brani presenti nella libreria in base a parole chiave riferite ai metadati principali del brano.	Fatto	VITTORIO POSTIGLIONE
		Task 3.3.2 - Test e validazione ricerca	TEST	Verificare che la ricerca mostri i brani corrispondenti, gestisca correttamente risultati vuoti e ripristini l'elenco completo quando il campo viene svuotato.	Fatto	ARIANNA PALETTA
US 1.10 - Salvataggio automatico dello stato della libreria	3					
		Task 1.10.1 - Implementazione salvataggio dati	CORE	Implementare la logica di persistenza locale per salvare automaticamente brani, playlist e metadati principali dell'applicazione in un file di salvataggio.	Fatto	MATTIA SANZARI
		Task 1.10.2 - Integrazione salvataggio alla chiusura dell'applicazione	CORE	Collegare il sistema di salvataggio al ciclo di vita dell'applicazione, in modo che i dati vengano aggiornati automaticamente alla chiusura.	Fatto	MATTIA SANZARI
		Task 1.10.3 - Test e validazione salvataggio	TEST	Verificare che il file di persistenza venga creato o aggiornato correttamente e che contenga i dati coerenti con lo stato della libreria.	Fatto	ARIANNA PALETTA
US 1.11 - Caricamento automatico della libreria all'avvio	1					
		Task 1.11.1 - Implementazione caricamento dati all'avvio	CORE	Implementare la logica che permette all'applicazione di leggere automaticamente il file di persistenza all'avvio e ricostruire la libreria musicale.	Fatto	MATTIA SANZARI
		Task 1.11.2 - Test e validazione caricamento iniziale	TEST	Verificare che l'applicazione carichi correttamente i dati salvati e che, in assenza di file di salvataggio, inizializzi una libreria vuota senza errori.	Fatto	MATTIA SANZARI
	34					

Burndown Chart e Velocity

- Completati **34/34 Story Points**, includendo funzionalità extra aggiunte.
- **Incremento delle Performance:** Velocity effettiva salita a 34 SP, segnando il valore più alto del progetto grazie a una gestione ottimale del carico di lavoro.
- **Chiusura del Backlog:** Attività portate a termine progressivamente ed eliminazione totale dei task entro l'ultimo giorno utile, senza colli di bottiglia.
- **Conclusione del Progetto:** Essendo l'ultimo sprint, i dati confermano la massima produttività e la maturità empirica raggiunte dal team nella gestione dei requisiti.



Retrospective Report

3rd Sprint Retrospective

STOP	LESS OF	KEEP DOING	MORE OF	START
Non sono emerse attività critiche da interrompere. Il team ha lavorato in modo coordinato e le scelte operative adottate durante la Sprint si sono rivelate efficaci.	Non sono emerse attività critiche da interrompere. Il team ha lavorato in modo coordinato e le scelte operative adottate durante la Sprint si sono rivelate efficaci.	Pianificazione flessibile supportata dalle sessioni in presenza, fondamentale per riorganizzare tempestivamente il backlog a fronte dell'inserimento di nuove storie. Allineamento continuo su Trello, rivelatosi determinante per monitorare l'avanzamento reale a fronte dell'aumento di scope complessivo.	Estensione delle sessioni di peer review per validare tempestivamente l'impatto dei nuovi Story Points inseriti sulla stabilità dell'architettura preesistente.	Non sono emerse nuove pratiche da introdurre nella Sprint successiva. Il team intende consolidare le modalità di lavoro già adottate, mantenendo continuità nell'organizzazione, nella comunicazione e nel tracciamento delle attività.

3rd Sprint Review

Velocity reale	34 / 34 SP completati. Considerato che nello sprint precedente il lavoro era stato terminato in anticipo, in questo sprint sono state incluse funzionalità extra, portando la velocity effettiva a 34 SP.
Sfida principale	La criticità maggiore è stata rappresentata dall'inserimento in corsa di 7 Story Points aggiuntivi. Questo incremento di scope ha richiesto una rapida riallocazione delle priorità per evitare colli di bottiglia e garantire il completamento entro la scadenza stabilita.
Calibrazione SP	L'esperienza acquisita negli sprint precedenti ha permesso al team di valutare meglio l'effort necessario per le attività pianificate. Nonostante alcune complessità progettuali, il carico complessivo dello sprint è stato gestito correttamente.
Stato al termine	Al termine dello Sprint 3 tutte le funzionalità previste e quelle inserite successivamente risultano rilasciate. La linea del burndown chart mostra un andamento che sale per registrare il nuovo scope e successivamente decresce in modo lineare fino al raggiungimento dello zero finale.

Prodotto Finale

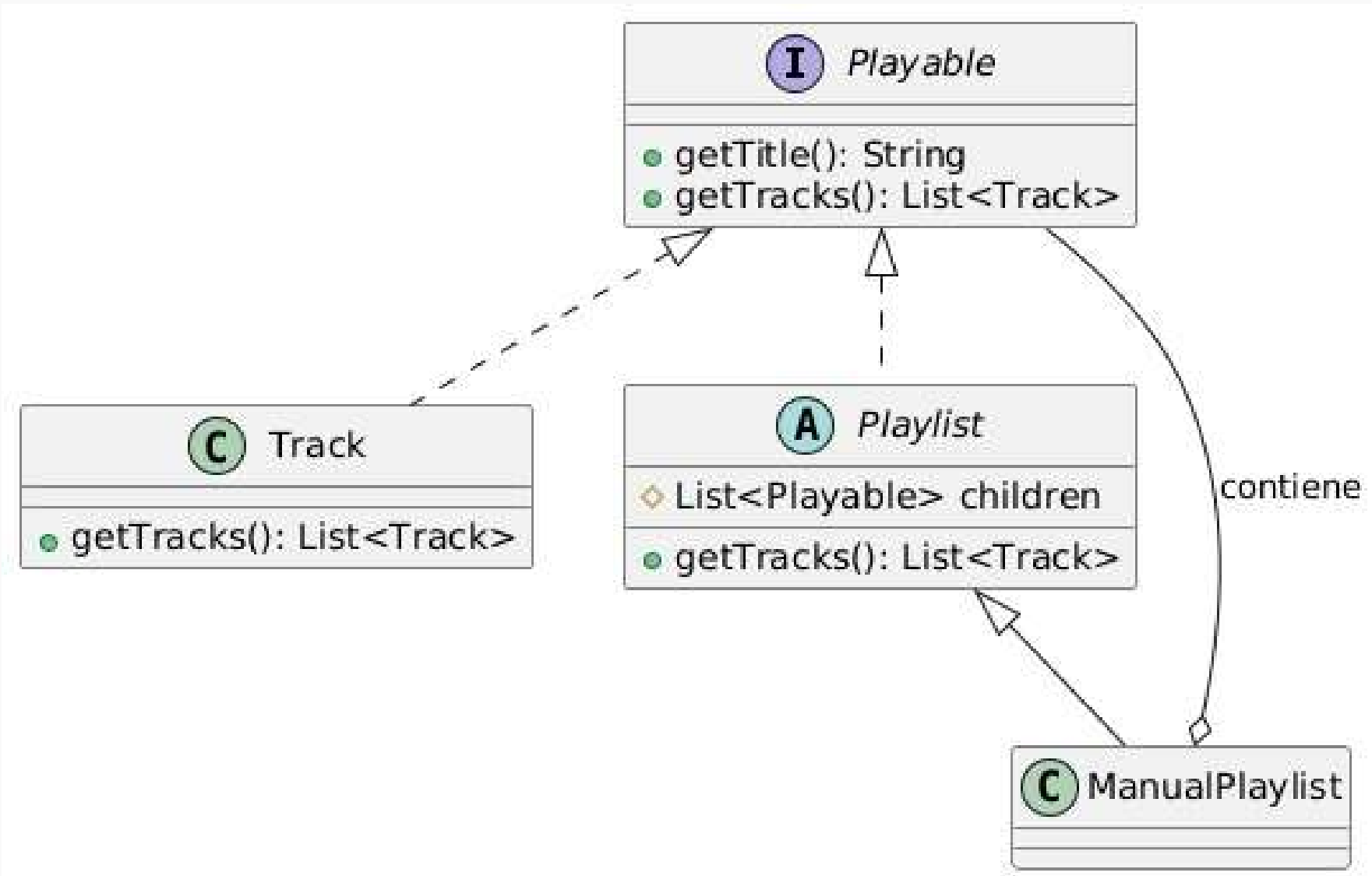


Design Patterns Utilizzati

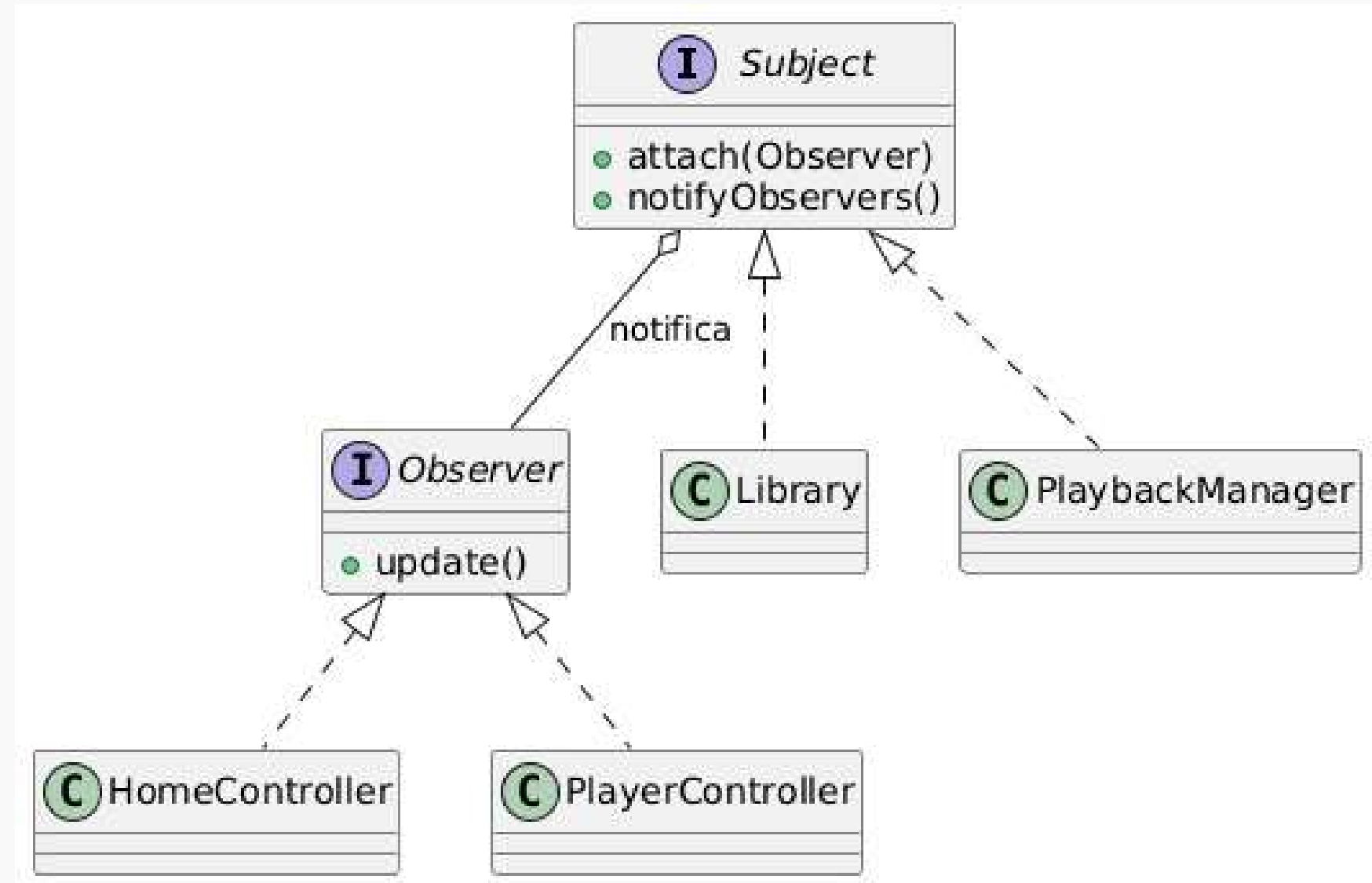
DESIGN PATTERN	CLASSI COINVOLTE	PROBLEMATICA RISOLTA
Singleton	Library, PlaybackManager, UndoManager, ThemeManager	Garantisce che esista una sola istanza globale per i gestori chiave dell'applicazione, evitando conflitti come l'esecuzione di due lettori audio simultanei o librerie desincronizzate.
Composite	Playable, Track, ManualPlaylist	Permette di trattare singoli oggetti (canzoni) e composizioni di oggetti (playlist) allo stesso modo. Il lettore può gestire code miste senza preoccuparsi della tipologia del nodo.
Factory Method	PlaylistGenerator, AutomaticPlaylistGenerator, ManualPlaylistGenerator, Playable	Disaccoppia la logica di creazione dal client, incapsulando la complessità legata all'istanziamento delle diverse tipologie di playlist (manuali, per anno, per tag, ecc.).
DAO (Data Access Object)	LibraryDAO, JsonLibraryDAO	Separa la complessa logica di serializzazione/deserializzazione JSON dalla logica di business. Nasconde ai controller i dettagli tecnici sul salvataggio e caricamento dei dati.
Command	Command, UndoManager, Library, AddTrackCommand, ECC.	Incapsula ogni azione dell'utente in un oggetto dedicato. Questo permette di memorizzare e accodare le richieste, rendendo possibile l'implementazione dell'operazione di Undo (Annulla).
State	PlaybackState, PlayingState, PausedState, StoppedState, PlaybackManager	Altera dinamicamente il comportamento interno del lettore in base allo stato attuale (es. l'effetto del tasto Play varia se si è in pausa o fermi), eliminando lunghe catene di costrutti if/else.
Strategy	PlaybackStrategy, SequentialStrategy, ShuffleStrategy, PlaybackManager, ECC.	Rende gli algoritmi per l'ordine di avanzamento della coda intercambiabili in tempo reale (Riproduzione Sequenziale, Casuale, Ripeti Singola, ecc.) con la pressione di un tasto nell'interfaccia.
Observer	Subject, Observer, Library, Track, PlaybackManager, Controller JavaFX	Permette all'interfaccia grafica (UI) di aggiornarsi in tempo reale ai cambiamenti del dominio (es. cambio canzone) senza creare dipendenze dirette e pericolose tra logica di business e vista.

DESIGN PATTERNS UTILIZZATI

COMPOSITE

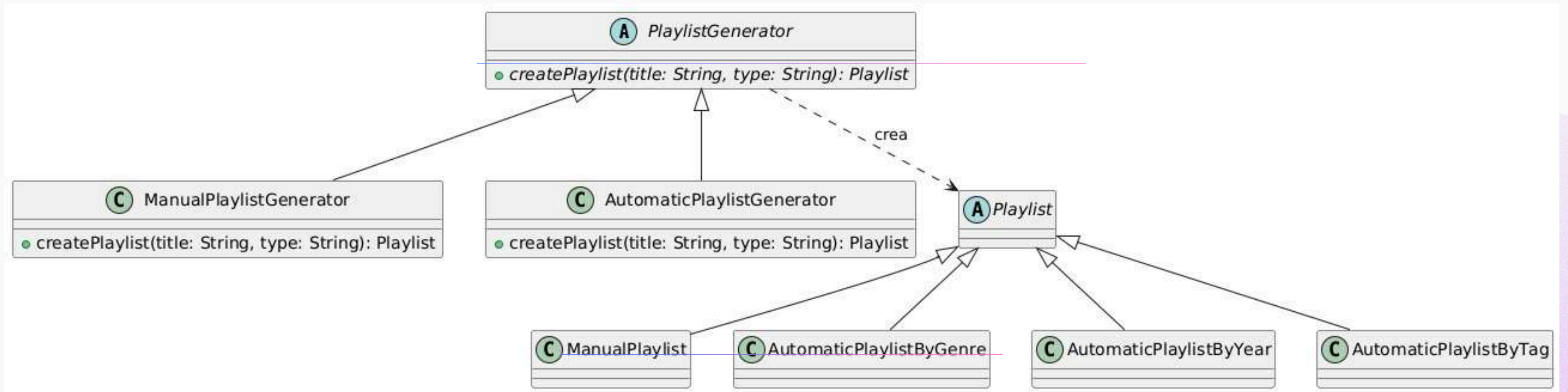


OBSERVER

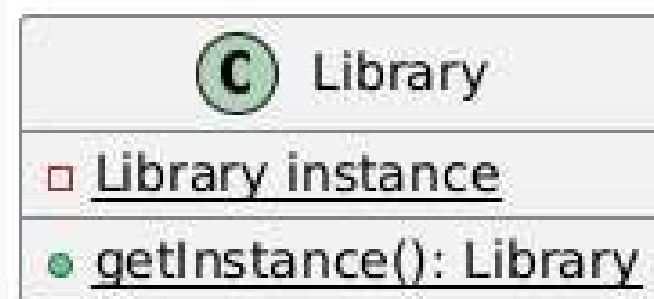


DESIGN PATTERNS UTILIZZATI

FACTORY METHOD

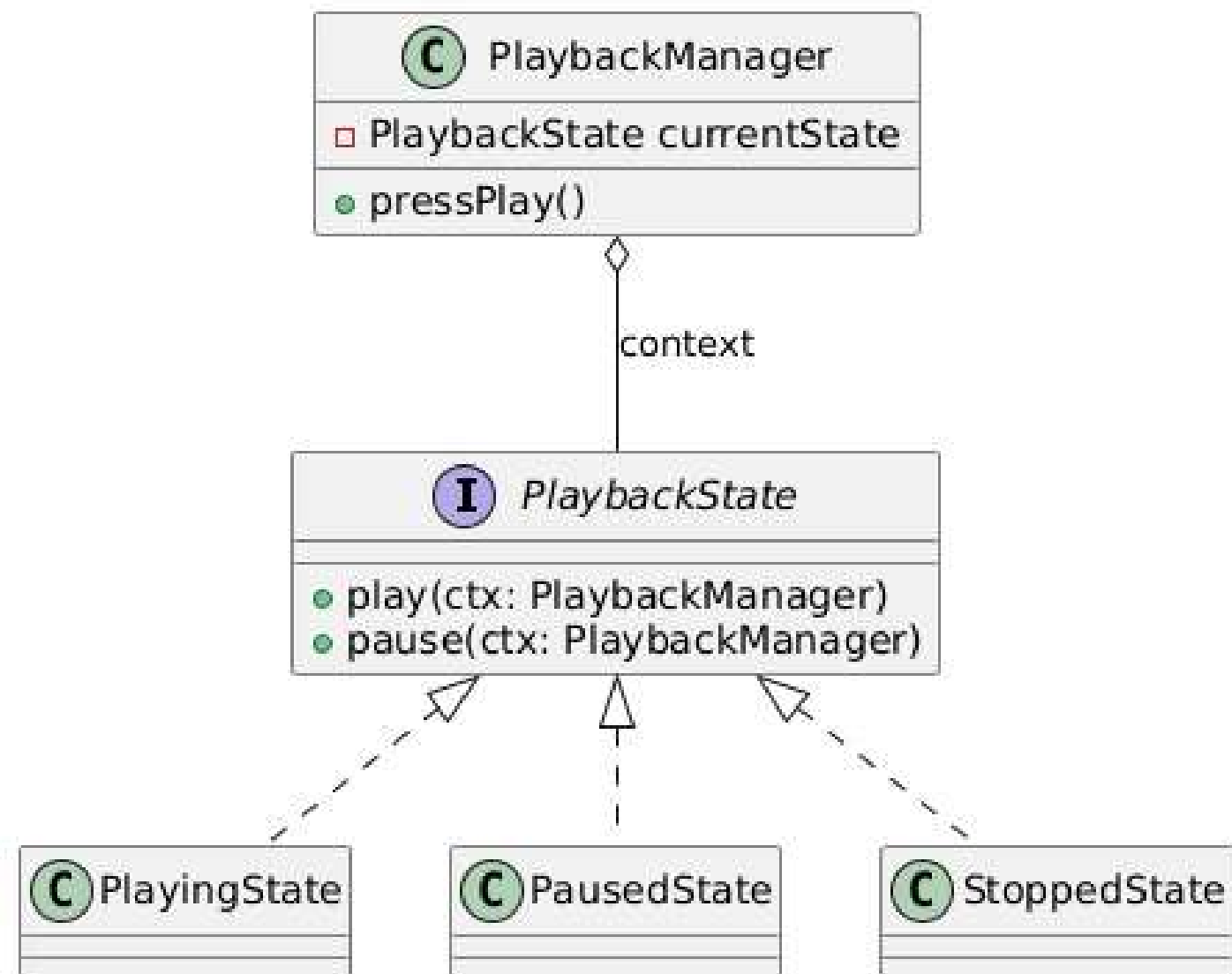


SINGLETON

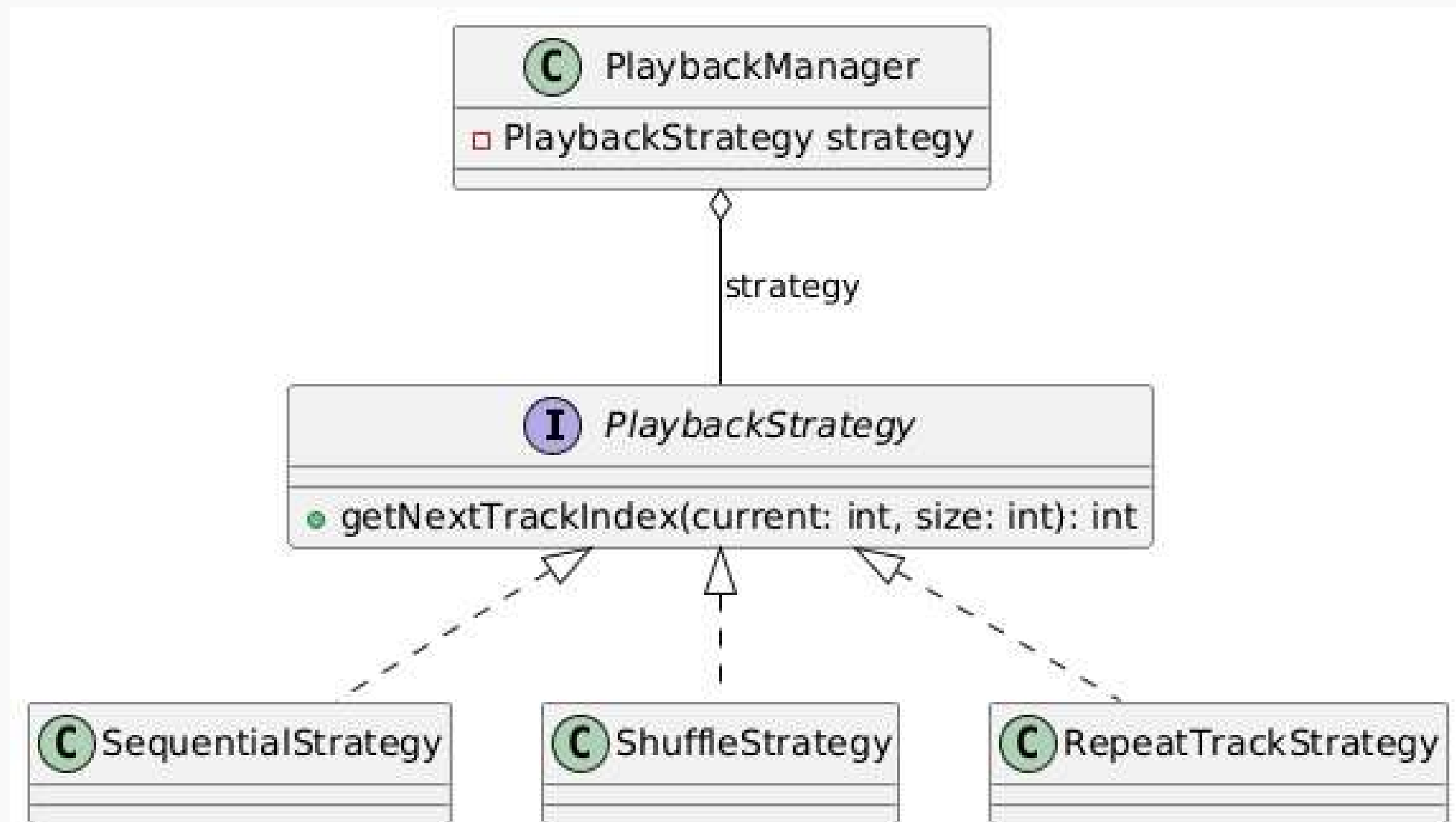


DESIGN PATTERNS UTILIZZATI

STATE

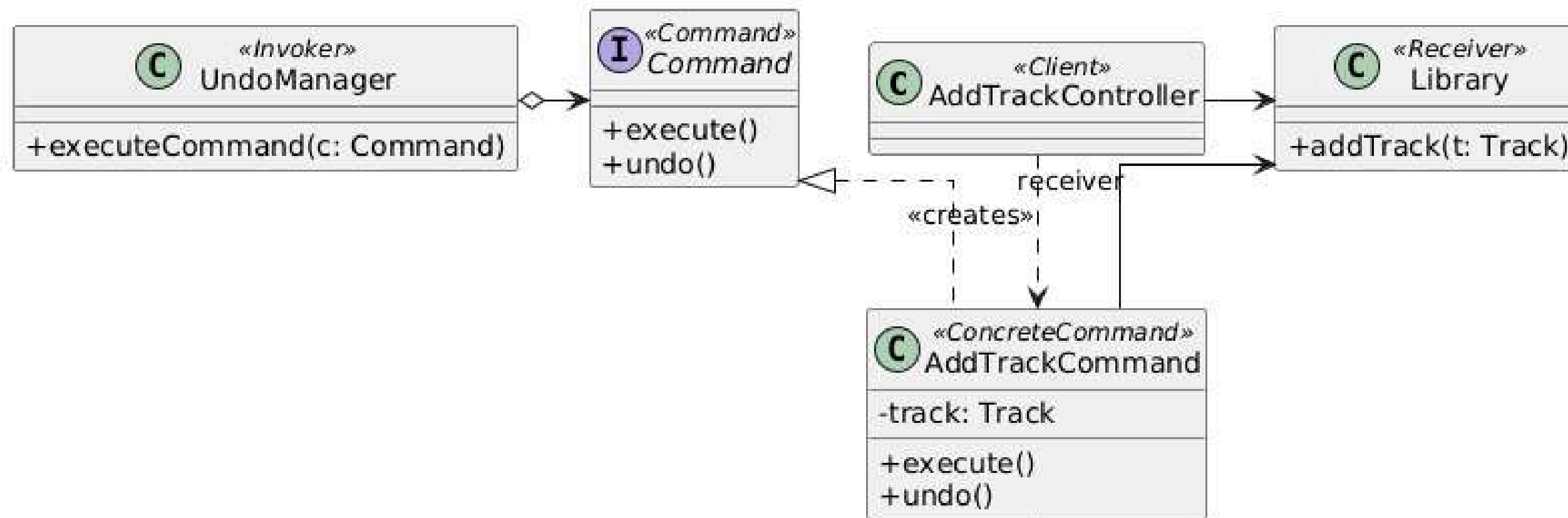


STRATEGY

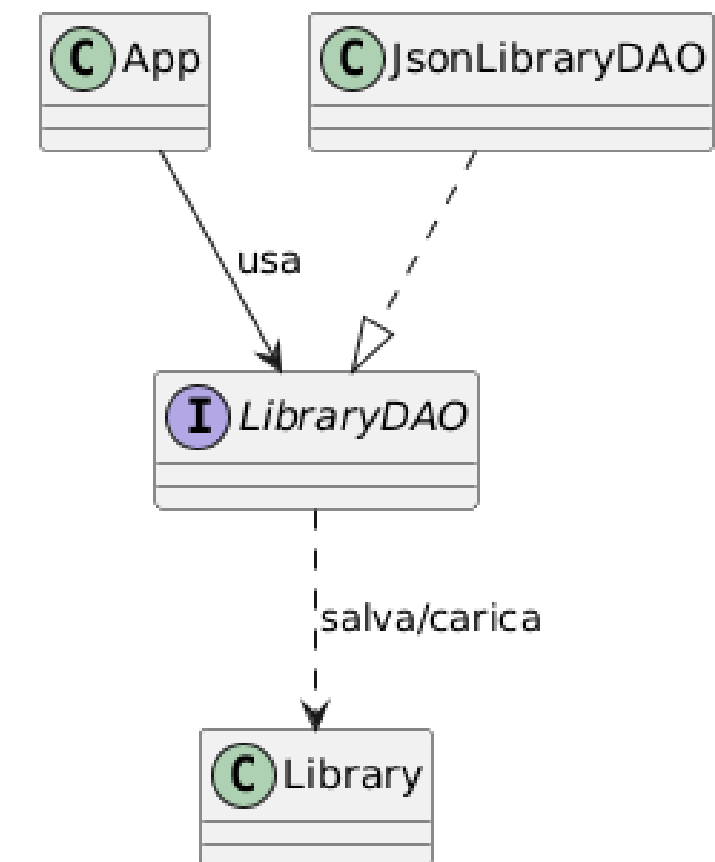


DESIGN PATTERNS UTILIZZATI

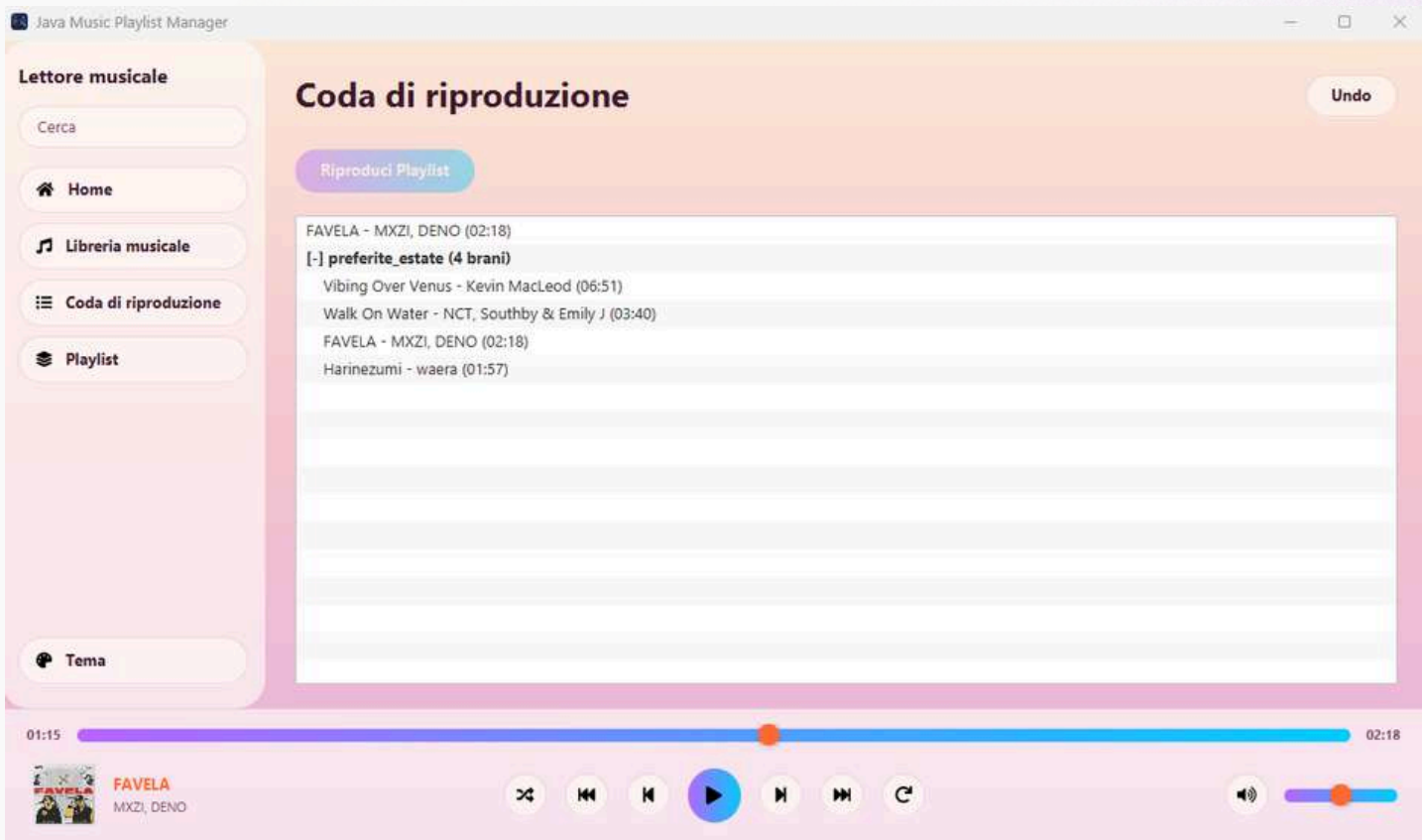
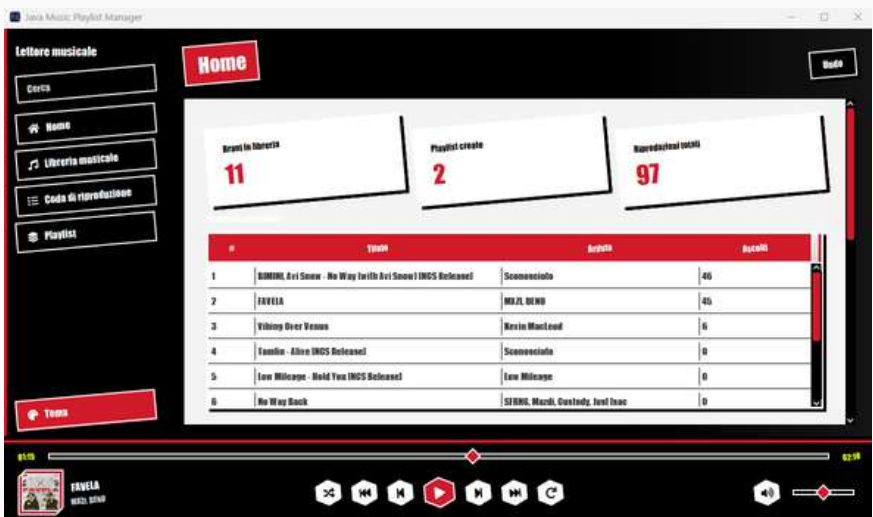
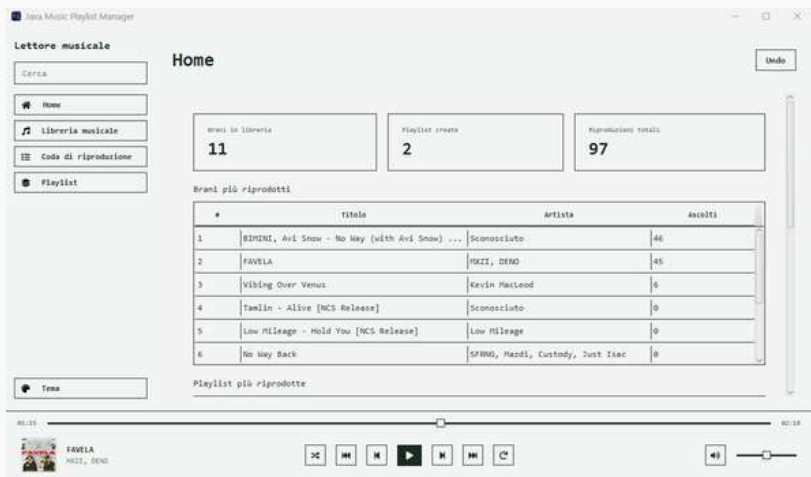
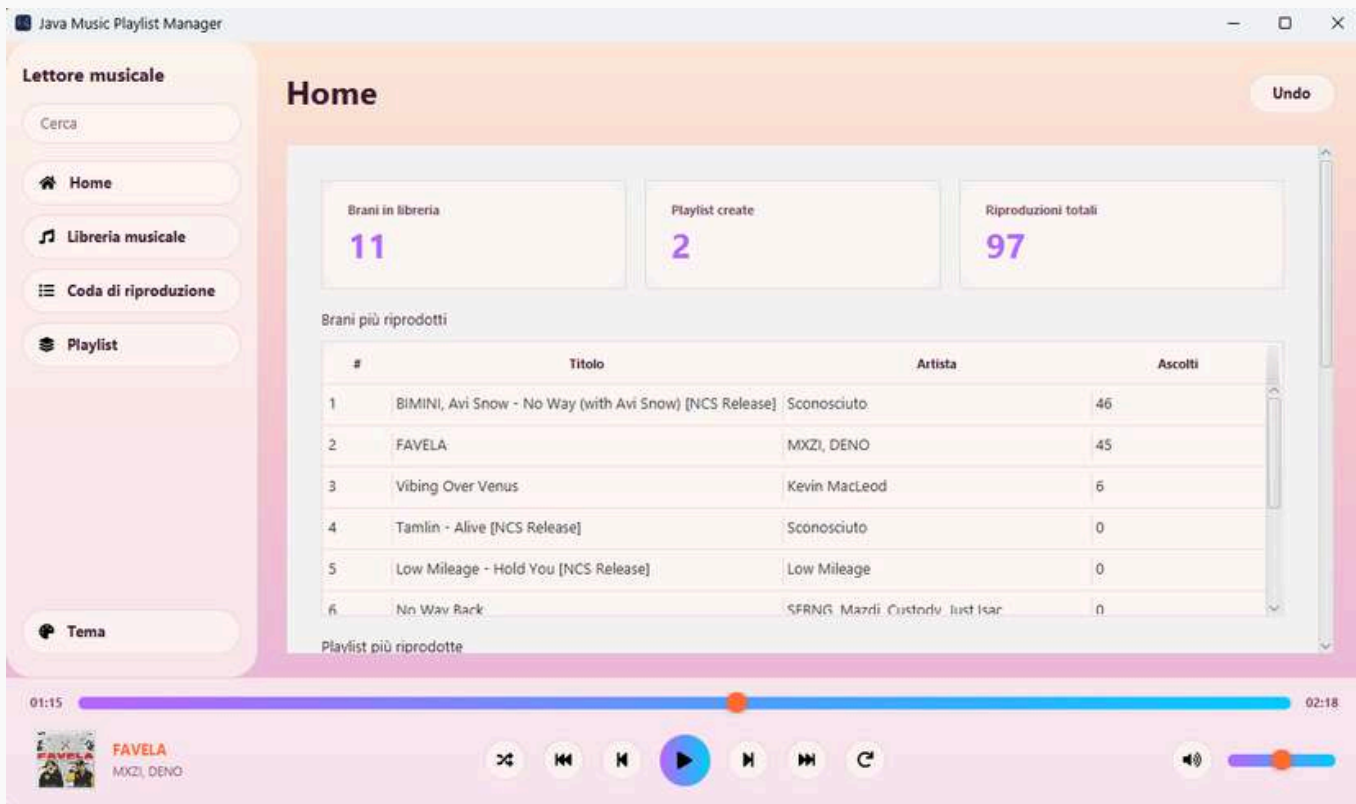
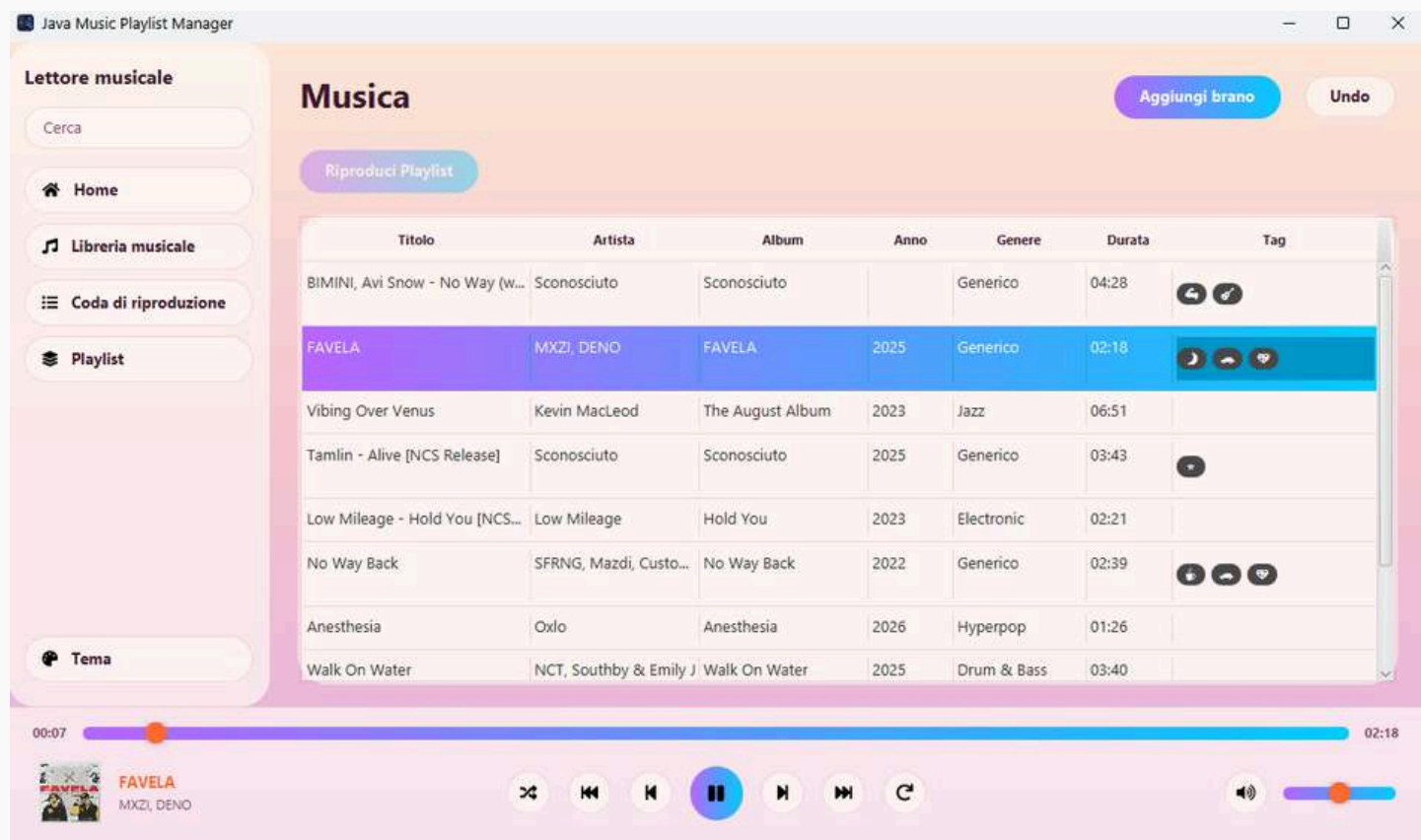
COMMAND



DAO



User Interface





Grazie per l'attenzione!

Gruppo 01

0622702428 – Domenico Nappo

IE22700035 – Arianna Paletta

IE22700189 – Vittorio Postiglione

IE22700199 – Mattia Sanzari

IE22700027 – Debora Villano